

Objetivo: Desenvolver, em formato de Sprint de 1 semana, um serviço web simples usando Django. O foco principal é empacotar essa aplicação em um container utilizando um Dockerfile e orquestrar a sua execução através de um docker-compose.yml.

Modalidade: Individual.

O Framework Ágil: Cronograma da Sprint (1 Semana)

Você trabalhará simulando o papel de um Desenvolvedor em uma equipe Ágil. Crie um quadro Kanban no GitHub Projects com as colunas: **To Do** (A Fazer), **Doing** (Fazendo) e **Done** (Feito). Distribua as tarefas abaixo ao longo dos dias.

Segunda-feira (Sprint Planning & Setup):

- Criar o repositório no GitHub.
- Criar o quadro Kanban e cadastrar as tarefas da semana.
- Configurar o ambiente virtual na máquina local.

Terça e Quarta-feira (Desenvolvimento do Serviço):

- Instalar o Django e iniciar o projeto web.
- Criar uma página (View) simples de "Catálogo de Produtos".
- Garantir que a aplicação roda localmente com o banco de dados padrão (SQLite).

Quinta-feira (Containerização & Orquestração):

- Escrever o `Dockerfile` isolando as dependências do serviço.
- Escrever o `docker-compose.yml` para automatizar a subida deste único serviço web.

Sexta-feira (Sprint Review & Entrega):

- Testar o comando de orquestração final.
- Escrever a documentação no `README.md`.
- Realizar a entrega final no repositório.

Passo a Passo da Atividade (Tarefas Técnicas)

Garante que cada um dos passos abaixo seja registrado e movimentado no seu quadro Kanban durante a semana.

Passo 1: Configuração do Repositório

- Crie um repositório público no GitHub com o nome `servico-django-docker`.
- Adicione um arquivo `.gitignore` focado na linguagem Python.
- Faça o clone para a sua máquina local.

Passo 2: O Serviço em Django

- Crie um ambiente virtual usando o comando `python -m venv venv` e ative-o.
- Instale o Django com o comando `pip install django`.
- Inicie o projeto executando `django-admin startproject core .`
- Execute as migrações iniciais para gerar o banco de dados local SQLite rodando `python manage.py migrate`.
- Crie uma View simples no Django (ou use a página inicial padrão) apenas para confirmar que o servidor web está respondendo.
- Gere o arquivo de dependências executando `pip freeze > requirements.txt`.

Passo 3: A Criação do Dockerfile

- Crie um arquivo chamado `Dockerfile` (exatamente assim, sem extensão) na raiz do seu projeto.
- Defina a imagem base utilizando `FROM python:3.10-slim`.
- Defina o diretório de trabalho interno com `WORKDIR /app`.
- Copie o arquivo de dependências usando `COPY requirements.txt .` e instale-as com `RUN pip install -r requirements.txt`.
- Copie o restante do código do seu projeto para o container com `COPY . .`
- Libere o acesso à porta do Django usando `EXPOSE 8000`.
- Defina o comando de inicialização padrão: `CMD ["python", "manage.py", "runserver", "0.0.0.0:8000"]`.

Passo 4: Orquestração com Docker Compose

- Crie um arquivo chamado `docker-compose.yml` na raiz do projeto.
- Defina a versão do compose (ex: `version: '3.8'`).
- Abra o bloco `services:` e crie o nosso único serviço, chamando-o de `web`.
- Dentro da configuração do serviço `web`, adicione a instrução `build: .` para ele ler o seu Dockerfile.
- Mapeie a porta de acesso local para a porta do container usando `ports: - "8000:8000"`.

Passo 5: Testes e Entrega

- No terminal, pare qualquer servidor Django que esteja rodando localmente.
- Execute o comando `docker-compose up --build`.
- Acesse o endereço `http://localhost:8000` no seu navegador e verifique se o seu serviço web está no ar sendo servido pelo Docker.
- Edite o seu arquivo `README.md` explicando os pré-requisitos, como rodar a aplicação usando o Docker Compose e insira um print do seu quadro Kanban.

- Faça o *push* do seu código para o GitHub.
- Colocar o link do repositório como resposta na atividade.